后端+AI推荐系统+C++底层优化 超详细全栈学习路线

（适配你的C++基础，全程无略写，含学习方法、资料、实战、通关标准）

前置说明

你的情况分析

- **已有基础**：C++编程基础（跳过纯小白语法）
- **核心目标**：掌握「后端工程开发（Python+Go双栈）+ AI推荐系统全链路 + C++底层性能优化」的复合能力，就业导向
- **学习节奏**：按每天学习2-3小时计算，总计18-22周（4.5-5.5个月）可达到入门就业水平，C++模块与主线并行/穿插，不拖慢进度
- **技术栈分工**：
  - **Python**：数据处理、特征工程、推荐算法训练、模型部署、业务逻辑开发
  - **Go**：高并发推荐召回/精排接口、实时日志处理、实时特征更新
  - **C++**：高性能向量召回引擎、模型推理加速、底层工具开发（进阶加分项）

第一阶段：编程&数据基础打底 + C++快速复习（3-4周）

阶段目标

完成后端+AI开发的基础能力铺垫，同时快速复习C++核心语法并入门现代C++

主线任务1：Python基础（1.5周，有C++基础可倍速）

学习重点

跳过「变量、循环、条件判断、函数定义」等通用编程概念，重点学Python特有内容：

1. Python数据结构：列表、字典、元组、集合的高级用法
2. 函数进阶：默认参数、可变参数、关键字参数、lambda表达式、装饰器
3. 面向对象：类、继承、多态、属性访问控制

4. 模块与包：导入、第三方库安装（pip）
5. IO操作：文件读写、CSV/Excel处理（pandas基础入门）
6. 数据库操作：Python连接PostgreSQL/MySQL（psycopg2/pymysql）
7. 网络请求：requests库的使用

学习方法

1. 先看廖雪峰教程的核心章节，每学完一个章节就写对应的小脚本
2. 遇到语法疑问直接查Python官方文档
3. 同步刷LeetCode简单题熟悉Python语法逻辑

核心必学资料

1. **廖雪峰Python3教程（网页版）**：只看以下模块，其余跳过
   - Python基础：https://www.liaoxuefeng.com/wiki/1016959663602400/1017024381818112
   - 函数：https://www.liaoxuefeng.com/wiki/1016959663602400/1017024645151424
   - 高级特性：https://www.liaoxuefeng.com/wiki/1016959663602400/1017024789634496
   - 面向对象编程：https://www.liaoxuefeng.com/wiki/1016959663602400/1017497232044640
   - 模块：https://www.liaoxuefeng.com/wiki/1016959663602400/1017498056865984
   - IO编程：https://www.liaoxuefeng.com/wiki/1016959663602400/1017498376963936
   - 进程和线程：https://www.liaoxuefeng.com/wiki/1016959663602400/1017631911925312
2. **Python官方中文文档**：当字典查，遇到疑问直接搜索
   - 地址：https://docs.python.org/zh-cn/3/
3. **Python requests库官方文档**：快速过一遍核心用法
   - 地址：https://docs.python-requests.org/zh_CN/latest/

补充参考资料

1. B站黑马程序员《Python入门到精通》：只看「Python操作数据库」「Python网络请求」「Pandas基础」模块，不用全刷
   - 地址：https://www.bilibili.com/video/BV1wD4y1o7AS/

必做实战任务

1. 用Python写一个**用户行为数据清洗脚本**：
   - 读取MovieLens开源数据集的CSV文件（https://grouplens.org/datasets/movielens/latest/）
   - 完成数据过滤（去除缺失值、异常值）、统计分析（统计每个用户的点击次数、每部电影的评分次数）

    - 输出清洗后的CSV文件
2. 用Python写一个**数据库操作脚本**：
    - 连接本地PostgreSQL数据库
    - 创建用户表、电影表、评分表
    - 插入清洗后的MovieLens数据
    - 实现简单的查询功能（查询某用户的所有评分、查询某电影的平均评分）
3. 用Python写一个**网络请求脚本**：
    - 用requests库调用一个公开的API（如天气API）
    - 解析返回的JSON数据，输出需要的信息
4. **LeetCode刷题**：刷完简单难度前20道题（用Python实现），熟悉Python语法逻辑
    - 地址：https://leetcode.cn/problemset/all/?difficulty=EASY&page=1

通关标准

1. 能独立用Python完成数据清洗、数据库操作、网络请求的小脚本
2. 能看懂Python的面向对象代码，会写简单的类
3. 能熟练使用pip安装第三方库
4. LeetCode简单题前20道能独立完成

主线任务2：SQL基础（1周，重点抓业务取数）

学习重点

不用学DBA的复杂运维内容，重点学后端+AI推荐取数必备的能力：

1. 基础SQL：增删改查（INSERT、UPDATE、DELETE、SELECT）
2. 条件查询：WHERE、LIKE、IN、BETWEEN、AND/OR
3. 排序与分页：ORDER BY、LIMIT、OFFSET
4. 聚合查询：COUNT、SUM、AVG、MAX、MIN、GROUP BY、HAVING
5. 联表查询：INNER JOIN、LEFT JOIN、RIGHT JOIN、自连接
6. 子查询：WHERE子查询、FROM子查询
7. 索引基础：索引的作用、创建索引、索引的优缺点
8. 事务基础：ACID特性、BEGIN、COMMIT、ROLLBACK

学习方法

1. 先看《SQL必知必会》，一周内看完，边看边在本地数据库练习

2. 同步用SQLZoo做交互式练习，巩固语法
3. 刷LeetCode SQL题库，掌握业务SQL写法

核心必学资料

1. **《SQL必知必会》（第5版）**：电子书/纸质书，薄、全是实战干货，一周可看完
   - 重点看：第1-18章（覆盖所有核心SQL语法）
2. **廖雪峰SQL教程（网页版）**：快速过一遍基础语法，和《SQL必知必会》搭配
   - 地址：https://www.liaoxuefeng.com/wiki/1177760294787200
3. **SQLZoo**：免费在线SQL练习平台，交互式练习，不用自己搭环境
   - 地址：https://sqlzoo.net/wiki/SQL_Tutorial
   - 必做：SELECT from WORLD、SELECT from Nobel、SELECT within SELECT、SUM and COUNT、JOIN、Using Null、Self join 这几个模块

补充参考资料

1. B站尚硅谷《MySQL入门到精通》：不用全刷，只看「索引、事务、存储引擎」这三个模块（面试必问）
   - 地址：https://www.bilibili.com/video/BV12b411K7Zu/

必做实战任务

1. **SQLZoo练习**：完成上面列出的所有必做模块
2. **LeetCode SQL刷题**：刷完简单+中等难度前20道题
   - 地址：https://leetcode.cn/problemset/database/
3. **MovieLens数据SQL练习**：
   - 用SQL查询：评分最高的10部电影、最活跃的10个用户、某用户评分最高的5部电影
   - 用SQL统计：每个类型的电影数量、每个用户的平均评分、每月的评分数量

通关标准

1. 能独立编写业务场景下的SQL查询（包括联表、聚合、子查询）
2. 能理解索引的作用，会创建简单的索引
3. 能理解事务的ACID特性
4. LeetCode SQL简单+中等前20道能独立完成

主线任务3：Git版本控制（3天，全是实战操作）

学习重点

不用学复杂的底层原理，重点学个人开发、团队协作必备的常用命令：

1. Git安装与配置：安装Git、配置用户名和邮箱
2. 本地仓库操作：初始化仓库、add、commit、status、log、reset
3. 分支管理：branch、checkout、switch、merge、rebase（基础）、冲突解决
4. 远程仓库操作：clone、push、pull、fetch、remote
5. GitHub/Gitee使用：注册账号、创建仓库、提交代码、Pull Request（PR）基础

学习方法

1. 先看廖雪峰Git教程，2天内看完，边看边在本地操作
2. 注册GitHub/Gitee账号，把之前写的Python脚本、SQL练习代码提交到仓库
3. 试着给一个开源项目提一个简单的PR（比如改个文档错别字）

核心必学资料

1. **廖雪峰Git教程（网页版）**：全程免费，一步一步带操作
   - 地址：https://www.liaoxuefeng.com/wiki/896043488025088
   - 必看：所有章节（全是日常工作必用内容）
2. **Git官方中文文档**：权威参考，遇到命令疑问直接查
   - 地址：https://git-scm.com/book/zh/v2

补充参考资料

1. **GitHub官方Git入门指南**：快速了解GitHub的使用
   - 地址：https://guides.github.com/introduction/git-handbook/

必做实战任务

1. **本地仓库练习**：
   - 在本地初始化一个Git仓库
   - 创建一个README.md文件，add并commit
   - 修改[README.md](README.md)，再次commit
   - 用log查看提交历史，用reset回退到上一个版本
2. **分支管理练习**：
   - 创建一个dev分支，切换到dev分支
   - 在dev分支上修改代码，commit
   - 切换回main分支，合并dev分支
   - 模拟冲突：在main和dev分支上修改同一个文件的同一行，解决冲突后合并

3. **远程仓库练习**：
   - 注册GitHub/Gitee账号
   - 创建一个新的远程仓库
   - 把本地仓库的代码push到远程仓库
   - 从远程仓库clone到另一个本地目录，修改代码后push回去
4. **提交之前的代码**：把第一阶段写的Python脚本、SQL练习代码都提交到GitHub/Gitee仓库，写好commit message

通关标准

1. 能熟练使用Git的本地仓库操作命令（add、commit、log、reset）
2. 能熟练使用Git的分支管理命令（branch、checkout、merge、冲突解决）
3. 能熟练使用Git的远程仓库操作命令（clone、push、pull）
4. 有自己的GitHub/Gitee仓库，里面有第一阶段的练习代码

并行C++任务：C++核心语法快速复习+现代C++入门（1.5周）

学习重点

跳过纯小白语法，重点复习C++核心并入门现代C++：

1. **C++核心语法快速回顾**（3天）：
   - 类与对象：构造函数、析构函数、拷贝构造函数、赋值运算符重载、this指针、访问控制（public/protected/private）
   - 继承与多态：public继承、虚函数、纯虚函数、抽象类、虚析构函数
   - STL容器：vector、list、deque、map、unordered_map、set、unordered_set的常用操作
   - STL算法：sort、find、count、for_each等常用算法
   - 指针与引用：指针运算、引用的使用、const指针/引用
   - 函数重载、运算符重载
2. **现代C++（C++11/14/17）核心特性**（1周）：
   - 智能指针：shared_ptr、unique_ptr、weak_ptr的使用，避免内存泄漏
   - lambda表达式：语法、捕获列表、使用场景
   - 移动语义：右值引用、std::move、移动构造函数、移动赋值运算符
   - auto/decltype：自动类型推导
   - 范围for循环：for (auto &x : container)

- std::thread基础：创建线程、join、detach
- std::function、std::bind

3. **C++编码规范**（0.5天）：Google C++ Style Guide，养成良好的编码习惯

学习方法

1. 先看《C++ Primer》第5版的核心模块，复习C++基础
2. 再看《Effective Modern C++》，学习现代C++特性
3. 每学一个特性就写对应的小代码练习
4. 用Google C++ Style规范检查自己的代码

核心必学资料

1. **《C++ Primer》（第5版）**：电子书/纸质书，只挑以下模块复习，不用全看
   - 第12-16章：类与对象、继承与多态
   - 第9-11章：STL容器、算法
   - 第18-19章：特殊工具与技术（选看）
   - 附录A：语言特性（快速查阅C++11新特性）
2. **《Effective Modern C++》**：电子书/纸质书，现代C++必看，重点看前25条
   - 必看：条款1-4（类型推导）、条款7-10（智能指针）、条款13-17（lambda、移动语义）
3. **Google C++ Style Guide**：网页版，养成良好的编码习惯
   - 地址：https://google.github.io/styleguide/cppguide.html
   - 重点看：命名约定、注释、格式、类、函数、STL使用
4. **C++官方中文参考**：当字典查，遇到语法疑问直接搜索
   - 地址：https://zh.cppreference.com/

补充参考资料

1. B站尚硅谷《C++入门到精通》：只看「C++11新特性」模块，不用全刷
   - 地址：https://www.bilibili.com/video/BV1bE411T7oZ/

必做实战任务

1. **用C++写一个用户行为数据统计工具**：
   - 读取MovieLens数据集的CSV文件
   - 用STL容器（unordered_map存储用户数据、vector存储电影数据）
   - 完成数据过滤、统计分析（统计每个用户的点击次数、每部电影的平均评分）
   - 输出统计结果

2. **用现代C++重写上面的工具**：
    - 用智能指针管理内存
    - 用lambda表达式配合STL算法（for_each、sort）
    - 用移动语义避免不必要的拷贝
    - 用范围for循环遍历容器
3. **用std::thread写一个简单的多线程程序**：
    - 创建多个线程，每个线程统计一部分数据
    - 用join等待所有线程完成
    - 合并统计结果
4. **代码规范检查**：用Google C++ Style规范检查上面的代码，修改不符合规范的地方
5. **提交代码**：把C++练习代码提交到GitHub/Gitee仓库，写好commit message

通关标准

1. 能熟练使用C++的类与对象、继承多态、STL容器与算法
2. 能熟练使用现代C++的核心特性（智能指针、lambda、移动语义、auto、范围for）
3. 能写简单的多线程程序
4. 代码符合Google C++ Style规范
5. GitHub/Gitee仓库里有C++练习代码

第二阶段：后端核心能力搭建 + C++网络编程（6-7周）

阶段目标

能独立开发高并发后端接口、熟练使用核心数据库&中间件、完成容器化部署，同时学习C++网络编程和中间件客户端

主线任务1：Python FastAPI后端开发（2周）

学习重点

FastAPI是高性能异步Web框架，是AI推荐接口的首选，重点学：

1. FastAPI基础：安装、创建第一个应用、运行服务
2. 路由与请求处理：路径参数、查询参数、请求体（JSON）、表单数据、文件上传
3. 参数验证：Pydantic模型、字段验证、必填/可选参数

4. 响应处理：JSON响应、状态码、响应模型、错误处理（HTTPException）
5. 中间件：自定义中间件、CORS中间件（跨域处理）
6. 数据库连接：SQLAlchemy ORM连接PostgreSQL、数据库迁移（Alembic基础）
7. 依赖注入：Depends的使用（数据库连接依赖、认证依赖基础）
8. 接口文档：自动生成的Swagger/OpenAPI文档

学习方法

1. 先看FastAPI官方中文文档，边看边写代码
2. 同步看B站的FastAPI入门视频，补充理解
3. 完成一个完整的用户管理、行为日志上报的后端接口项目

核心必学资料

1. **FastAPI官方中文文档**：最权威的资料，全程跟着做
   - 地址：https://fastapi.tiangolo.com/zh/
   - 必看：
     - 教程 - 用户指南：https://fastapi.tiangolo.com/zh/tutorial/
     - 进阶用户指南：https://fastapi.tiangolo.com/zh/advanced/（选看中间件、依赖注入、数据库部分）
2. **SQLAlchemy官方文档**：快速过一遍ORM的核心用法
   - 地址：https://docs.sqlalchemy.org/en/20/orm/
3. **Pydantic官方文档**：快速过一遍模型定义和验证
   - 地址：https://docs.pydantic.dev/latest/

补充参考资料

1. B站《FastAPI入门到精通》：选看「FastAPI基础、数据库连接、中间件」模块
   - 地址：https://www.bilibili.com/video/BV1s44y1g7QV/

必做实战任务

1. **FastAPI基础练习**：
   - 创建一个FastAPI应用，实现简单的GET/POST请求
   - 用路径参数、查询参数、请求体接收数据
   - 用Pydantic模型做参数验证
   - 返回JSON响应，设置状态码
   - 访问自动生成的Swagger文档（http://127.0.0.1:8000/docs）测试接口

2. **用户管理&行为日志上报接口项目**：
    - 用SQLAlchemy连接PostgreSQL，创建用户表、行为日志表
    - 实现用户注册、登录（基础的密码验证，不用复杂认证）、用户信息查询接口
    - 实现行为日志上报接口（接收用户的点击、浏览、收藏等行为，存入数据库）
    - 用Alembic做简单的数据库迁移（创建表、修改表结构）
    - 加CORS中间件，允许跨域请求
    - 用Swagger文档测试所有接口
3. **提交代码**：把FastAPI项目代码提交到GitHub/Gitee仓库，写好[README.md](README.md)（说明项目功能、如何运行）

通关标准

1. 能独立用FastAPI开发RESTful接口
2. 能熟练使用Pydantic做参数验证
3. 能熟练使用SQLAlchemy连接数据库、做CRUD操作
4. 能使用FastAPI的中间件、依赖注入
5. 能访问Swagger文档测试接口
6. 有完整的FastAPI项目代码在GitHub/Gitee仓库

主线任务2：Go后端开发（2周，有C++基础可倍速到1.5周）

学习重点

Go是高并发推荐接口的首选，你的C++基础能快速上手，重点学：

1. **Go语言基础（1周，C++基础可缩到3-5天）**：
    - Go特有语法：包、变量、常量、基本数据类型、指针（简化版，和C++类似但更安全）
    - 结构体与方法：struct、method（类似C++的类成员函数）
    - 接口：interface（非侵入式，和C++的虚函数类似但更灵活）
    - 错误处理：error类型、defer、panic/recover
    - 切片与映射：slice（类似C++的vector但更强大）、map（类似C++的unordered_map）
    - Goroutine并发模型：goroutine（轻量级线程，比C++的std::thread更轻量）、Channel（用于goroutine之间的通信）、sync包（Mutex、WaitGroup）
2. **Go Web框架Gin（0.5周）**：
    - Gin基础：安装、创建第一个应用、运行服务

- 路由：GET/POST/PUT/DELETE、路径参数、查询参数、路由组
- 参数绑定：JSON、表单、路径参数绑定到结构体
- 响应处理：JSON响应、状态码、文件下载
- 中间件：自定义中间件、CORS中间件、日志中间件、恢复中间件
- 错误处理：统一错误处理

3. **Go对接核心中间件&数据库（0.5周）**：
   - Go Redis客户端：go-redis库，连接Redis、读写数据、实现简单的缓存、限流
   - Go PostgreSQL客户端：pgx库，连接PostgreSQL、做CRUD操作
   - Go Kafka客户端：sarama库，生产/消费消息（基础）

学习方法

1. 先看《Go语言圣经》的前10章，快速入门Go基础
2. 再看Gin官方文档，学习Gin框架
3. 同步看B站的Go入门视频，补充理解
4. 完成一个高并发推荐召回接口的雏形项目

核心必学资料

1. **《Go语言圣经》（The Go Programming Language）**：电子书/纸质书，只看前10章，C++基础看这个极快
   - 重点看：第1-5章（基础语法）、第6章（结构体与方法）、第7章（接口）、第8章（Goroutine与Channel）、第9章（并发编程）、第10章（包与工具）
2. **Go官方中文文档**：当字典查
   - 地址：https://go.dev/doc/
3. **Gin官方中文文档**：全程跟着做
   - 地址：https://gin-gonic.com/zh-cn/docs/
4. **go-redis官方文档**：快速过一遍核心用法
   - 地址：https://redis.uptrace.dev/zh/
5. **pgx官方文档**：快速过一遍核心用法
   - 地址：https://pkg.go.dev/github.com/jackc/pgx/v5
6. **sarama官方文档**：快速过一遍基础用法
   - 地址：https://github.com/IBM/sarama

补充参考资料

1. B站尚硅谷《Go语言入门到精通》：只看「Go基础-并发编程」「Gin框架」模块

- 地址：https://www.bilibili.com/video/BV1ME411Y71o/

必做实战任务

1. **Go基础练习**：
   - 写一个简单的Go程序，用结构体、方法、接口
   - 写一个多线程程序：用goroutine和Channel实现生产者-消费者模型
   - 用sync.Mutex实现线程安全的计数器
   - 用defer释放资源
2. **Gin基础练习**：
   - 创建一个Gin应用，实现简单的GET/POST请求
   - 用路径参数、查询参数、JSON请求体接收数据
   - 用结构体绑定参数
   - 返回JSON响应，设置状态码
   - 加一个简单的日志中间件，记录每个请求的时间和路径
3. **Go+Redis+PostgreSQL练习**：
   - 用go-redis连接本地Redis，实现简单的缓存：先查Redis，没有再查PostgreSQL，然后存入Redis
   - 用pgx连接本地PostgreSQL，做用户表的CRUD操作
4. **高并发推荐召回接口雏形项目**：
   - 用Gin创建一个推荐召回接口：接收用户ID，返回Top-10相似内容（暂时用模拟数据）
   - 用go-redis做限流：每个用户每分钟最多请求10次
   - 用goroutine处理并发请求，测试QPS（用ab或wrk压测工具）
5. **提交代码**：把Go练习代码和项目代码提交到GitHub/Gitee仓库，写好[README.md](README.md)

通关标准

1. 能熟练使用Go的基础语法、结构体、方法、接口
2. 能熟练使用Goroutine和Channel做并发编程
3. 能独立用Gin开发RESTful接口
4. 能熟练使用go-redis、pgx连接Redis和PostgreSQL
5. 能写简单的Gin中间件
6. 有完整的Go项目代码在GitHub/Gitee仓库

主线任务3：核心中间件&容器化部署（1.5周）

学习重点

1. **Redis（0.5周）**：
   - Redis数据结构：String、Hash、List、Set、Sorted Set的常用命令
   - Redis持久化：RDB、AOF（基础）
   - Redis应用场景：缓存、限流、排行榜、分布式锁（基础）、会话管理
   - Python/Go连接Redis：之前已经学过，这里重点学应用场景
2. **Kafka（0.5周）**：
   - Kafka基础概念：Broker、Topic、Partition、Producer、Consumer、Consumer Group
   - Kafka安装与配置：本地安装Kafka、创建Topic
   - Kafka常用命令：创建Topic、查看Topic列表、生产消息、消费消息
   - Python/Go连接Kafka：之前已经学过，这里重点学应用场景（用户行为日志采集、异步任务）
3. **Elasticsearch（0.3周）**：
   - Elasticsearch基础概念：Index、Document、Field、Node、Cluster
   - Elasticsearch安装与配置：本地安装Elasticsearch、Kibana
   - Elasticsearch常用操作：创建Index、插入Document、查询Document（Match Query、Term Query、Bool Query）
   - Python连接Elasticsearch：elasticsearch-py库
4. **Docker（0.2周）**：
   - Docker基础概念：Image、Container、Dockerfile
   - Docker常用命令：pull、push、build、run、stop、rm、logs、exec
   - Dockerfile编写：写一个简单的Dockerfile，打包FastAPI/Go应用
   - Docker Compose（基础）：用Docker Compose启动PostgreSQL、Redis、Kafka、Elasticsearch（简化本地环境搭建）

学习方法

1. 先看廖雪峰的Redis/Kafka教程，快速入门
2. 本地安装这些中间件，跟着文档做常用操作
3. 用Python/Go连接这些中间件，写小代码练习
4. 学习Docker，把之前的FastAPI/Go项目打包成镜像

核心必学资料

1. **廖雪峰Redis教程**：快速过一遍
   - 地址：https://www.liaoxuefeng.com/wiki/1252554847796928
2. **Redis官方中文文档**：当字典查
   - 地址：https://redis.io/docs/
3. **Kafka官方中文文档**：快速过一遍基础概念
   - 地址：https://kafka.apachecn.org/
4. **Elasticsearch官方中文指南**：快速过一遍基础操作
   - 地址：https://www.elastic.co/guide/cn/elasticsearch/guide/current/index.html
5. **廖雪峰Docker教程**：快速过一遍
   - 地址：https://www.liaoxuefeng.com/wiki/1252554847796928/1252555188488832
6. **Docker官方中文入门教程**：快速过一遍
   - 地址：https://docs.docker.com/get-started/

补充参考资料

1. **《Redis设计与实现》**：电子书/纸质书，面试必看，理解Redis的底层原理（选看，学有余力再看）
2. B站《Kafka入门到实战》《Elasticsearch入门到精通》《Docker+K8s入门到实战》：选看基础部分

必做实战任务

1. **Redis练习**：
   - 本地安装Redis，用redis-cli做常用操作：String的get/set、Hash的hget/hset、List的lpush/rpop、Set的sadd/smembers、Sorted Set的zadd/zrange/zrank
   - 用Python/Go实现一个简单的排行榜：用Sorted Set存储用户的分数，查询Top-10用户，查询某用户的排名
   - 用Python/Go实现一个简单的限流：用Redis的String类型，key是用户ID，value是请求次数，过期时间1分钟
2. **Kafka练习**：
   - 本地安装Kafka（用Docker Compose更简单），创建一个Topic（user_behavior）
   - 用Python/Go写一个Kafka生产者：模拟用户行为日志（用户ID、内容ID、行为类型、时间戳），发送到Topic
   - 用Python/Go写一个Kafka消费者：消费Topic里的消息，打印出来
3. **Elasticsearch练习**：
   - 本地安装Elasticsearch和Kibana（用Docker Compose）

- 创建一个Index（content），插入一些模拟的内容数据（内容ID、标题、描述、类型）
- 用Kibana的Dev Tools做查询：Match Query（按标题搜索）、Term Query（按类型过滤）、Bool Query（组合搜索和过滤）
- 用Python的elasticsearch-py库连接Elasticsearch，做同样的查询

4. **Docker练习**：
   - 写一个Dockerfile，打包之前的FastAPI项目：基础镜像用python:3.11-slim，安装依赖，复制代码，启动命令
   - 用docker build构建镜像，用docker run启动容器，测试接口
   - 写一个简单的docker-compose.yml文件，启动FastAPI容器、PostgreSQL容器、Redis容器，配置网络和 volumes
5. **提交代码**：把Dockerfile、docker-compose.yml提交到GitHub/Gitee仓库，更新之前的项目[README.md](README.md)，说明如何用Docker运行项目

通关标准

1. 能熟练使用Redis的常用数据结构和命令，能实现缓存、限流、排行榜
2. 能理解Kafka的基础概念，能生产/消费消息
3. 能理解Elasticsearch的基础概念，能做简单的查询
4. 能写Dockerfile打包应用，能用Docker Compose启动多个容器
5. 之前的FastAPI/Go项目能用Docker运行

并行C++任务：C++网络编程+中间件客户端（2周）

学习重点

1. **C++网络编程基础（1周）**：
   - Socket编程基础：TCP/UDP协议、socket()、bind()、listen()、accept()、connect()、send()、recv()
   - 简单的TCP服务器：多线程/多进程处理客户端请求（选学I/O多路复用：select/poll/epoll基础，学有余力再看）
   - 简单的HTTP服务器：解析HTTP请求、返回HTTP响应（JSON格式）
2. **中间件C++客户端（0.5周）**：
   - Redis C++客户端：hiredis（C语言库，C++可以直接用）或cpp_redis（C++封装库），连接Redis、读写数据
   - Kafka C++客户端：librdkafka（C语言库，C++可以直接用），生产/消费消息
3. **C++ JSON库（0.3周）**：nlohmann/json（现代C++ JSON库，非常好用），解析和生成JSON

4. **C++性能分析工具（0.2周）**：gprof、perf（基础），分析代码性能瓶颈

学习方法

1. 先看《UNIX网络编程 卷1》的核心模块，学习Socket编程
2. 再看hiredis、librdkafka、nlohmann/json的官方文档，学习中间件客户端和JSON库
3. 写小代码练习，完成一个简单的HTTP服务器和中间件客户端
4. 用gprof分析简单的C++程序性能

核心必学资料

1. **《UNIX网络编程 卷1：套接字联网API》（第3版）**：电子书/纸质书，只看以下模块，不用全看
   - 第1章：简介
   - 第3章：套接字编程简介
   - 第4章：基本TCP套接字编程
   - 第5章：TCP客户/服务器程序示例
   - 第6章：I/O复用：select和poll函数（选看）
   - 第11章：名字与地址转换
2. **hiredis官方文档**：快速过一遍核心用法
   - 地址：https://github.com/redis/hiredis
3. **librdkafka官方文档**：快速过一遍基础用法
   - 地址：https://github.com/confluentinc/librdkafka
4. **nlohmann/json官方文档**：快速过一遍核心用法
   - 地址：https://github.com/nlohmann/json
5. **gprof官方文档**：快速过一遍基础用法
   - 地址：https://sourceware.org/binutils/docs/gprof/

补充参考资料

1. B站《C++网络编程入门》：选看基础部分

必做实战任务

1. **C++ Socket编程练习**：
   - 写一个简单的TCP服务器：监听端口，接受客户端连接，接收客户端发送的消息，打印出来，然后回复消息
   - 写一个简单的TCP客户端：连接服务器，发送消息，接收服务器的回复，打印出来
2. **C++简单HTTP服务器**：

- 基于上面的TCP服务器，解析简单的HTTP GET请求
- 返回HTTP响应，状态码200，Content-Type为application/json，返回一个简单的JSON数据（比如{"message": "hello world"}）
- 用curl或浏览器测试这个HTTP服务器

3. **C++ Redis客户端练习**：
   - 用hiredis连接本地Redis
   - 实现String的get/set、Hash的hget/hset操作
   - 实现一个简单的缓存：先查Redis，没有的话返回默认值，然后存入Redis
4. **C++ Kafka客户端练习**：
   - 用librdkafka写一个Kafka生产者：连接本地Kafka，发送模拟的用户行为日志到user_behavior Topic
   - 用librdkafka写一个Kafka消费者：消费user_behavior Topic的消息，用nlohmann/json解析JSON数据，打印出来
5. **C++性能分析练习**：
   - 写一个简单的C++程序：比如对一个大数组进行排序
   - 用gprof编译并运行程序，生成性能分析报告，查看函数的执行时间
6. **提交代码**：把C++网络编程和中间件客户端的练习代码提交到GitHub/Gitee仓库，写好[README.md](README.md)

通关标准

1. 能理解Socket编程的基础概念，能写简单的TCP服务器和客户端
2. 能写一个简单的HTTP服务器，返回JSON响应
3. 能熟练使用hiredis连接Redis、做简单的操作
4. 能熟练使用librdkafka生产/消费Kafka消息
5. 能熟练使用nlohmann/json解析和生成JSON
6. 能用gprof分析简单的C++程序性能
7. GitHub/Gitee仓库里有对应的练习代码

第三阶段：AI推荐系统核心能力 + C++模型推理（7-8周）

阶段目标

理解工业级推荐系统全链路、能独立完成推荐算法的训练到部署、实现完整的个性化推荐功能，同时学习C++模型推理加速

主线任务1：数据处理&特征工程（1周）

学习重点

推荐系统80%的工作都在数据处理和特征工程，重点学：

1. **Pandas/NumPy核心用法**：
   - NumPy：数组创建、数组运算、索引与切片、常用函数（sum、mean、max、min、reshape）
   - Pandas：Series、DataFrame的创建、索引与切片、数据清洗（缺失值处理、异常值处理、重复值处理）、数据转换（apply、map、groupby、merge、concat）、数据可视化（Matplotlib/Seaborn基础，选看）
2. **特征工程基础**：
   - 特征类型：数值特征、类别特征、时间特征、文本特征（基础）
   - 特征处理：数值特征的归一化/标准化、类别特征的One-Hot编码/Label编码、时间特征的提取（年、月、日、星期、小时）
   - 特征选择：基础方法（相关性分析、方差阈值）
3. **用户/物品特征构建**：
   - 用户特征：用户的历史点击次数、历史平均评分、最近一次点击时间、偏好的内容类型
   - 物品特征：物品的历史点击次数、历史平均评分、发布时间、内容类型、内容标签
   - 上下文特征：请求时间、请求地点（基础）

学习方法

1. 先看《利用Python进行数据分析》的核心模块，学习Pandas/NumPy
2. 同步看B站的Pandas入门视频，补充理解
3. 用MovieLens数据集做数据处理和特征构建的练习

核心必学资料

1. **《利用Python进行数据分析》（第2版）**：电子书/纸质书，重点看以下模块
   - 第4章：NumPy基础：数组和矢量计算
   - 第5章：pandas入门
   - 第6章：数据加载、存储与文件格式
   - 第7章：数据清洗和准备
   - 第8章：数据规整：聚合、合并和重塑
   - 第10章：数据聚合与分组运算

2. **Pandas官方中文文档**：当字典查
   - 地址：https://pandas.pydata.org/docs/
3. **NumPy官方中文文档**：当字典查
   - 地址：https://numpy.org/doc/stable/

补充参考资料

1. B站《Pandas入门到实战》《NumPy入门到精通》：选看核心模块
2. **《特征工程入门与实践》**：电子书/纸质书，选看基础部分（学有余力再看）

必做实战任务

1. **Pandas/NumPy基础练习**：
   - 用NumPy创建数组，做数组运算、索引与切片
   - 用Pandas读取MovieLens数据集的CSV文件，创建DataFrame
   - 用Pandas做数据清洗：处理缺失值、去除重复值、过滤异常值
   - 用Pandas做数据转换：groupby统计用户的点击次数、merge合并用户表和电影表
2. **MovieLens特征构建练习**：
   - 构建用户特征：用户ID、历史点击次数、历史平均评分、最近一次点击时间、偏好的电影类型（用One-Hot编码或统计每个类型的点击次数）
   - 构建电影特征：电影ID、历史点击次数、历史平均评分、发布时间、电影类型（One-Hot编码）
   - 构建用户-电影交互特征：用户ID、电影ID、用户对电影的评分（标签）、用户点击电影的时间
   - 把构建好的特征保存为CSV文件，供后续算法训练使用
3. **提交代码**：把数据处理和特征构建的代码提交到GitHub/Gitee仓库，写好[README.md](README.md)

通关标准

1. 能熟练使用Pandas/NumPy做数据清洗、数据转换、数据统计
2. 能理解特征工程的基础概念，能处理数值特征、类别特征、时间特征
3. 能独立构建用户特征、物品特征、用户-物品交互特征
4. GitHub/Gitee仓库里有对应的练习代码

主线任务2：推荐系统基础&传统算法（2周）

学习重点

理解推荐系统的核心逻辑和全链路，学习传统推荐算法：

1. **推荐系统基础概念**：
   - 推荐系统的作用、应用场景（电商、内容、社交）
   - 推荐系统的核心链路：召回→粗排→精排→重排
   - 推荐系统的评价指标：离线指标（准确率、召回率、F1、RMSE、MAE）、在线指标（点击率CTR、转化率CVR、留存率、GMV）
   - 推荐系统的冷启动问题：用户冷启动、物品冷启动、解决方案基础
2. **传统推荐算法**：
   - 基于协同过滤的推荐（CF）：
     - 基于用户的协同过滤（User-CF）：找到相似的用户，推荐相似用户喜欢的物品
     - 基于物品的协同过滤（Item-CF）：找到相似的物品，推荐用户喜欢的物品的相似物品
     - 协同过滤的实现：相似度计算（余弦相似度、皮尔逊相关系数）、Top-N推荐
   - 基于内容的推荐（CB）：利用用户的历史行为和物品的内容特征（如电影的类型、演员）做推荐
   - 基于逻辑回归的推荐（LR）：把推荐问题转化为二分类问题（用户是否会点击/评分），用LR模型训练
   - 传统算法的优缺点对比

学习方法

1. 先看《推荐系统实践》，理解推荐系统的基础概念和传统算法
2. 同步看B站王树森的《推荐系统》公开课，补充理解
3. 用MovieLens数据集实现传统推荐算法，做离线评估

核心必学资料

1. **《推荐系统实践》**：项亮著，电子书/纸质书，推荐系统入门神书，必看
   - 重点看：第1章（好的推荐系统）、第2章（利用用户行为数据）、第3章（推荐系统冷启动）、第4章（利用用户标签数据）、第5章（利用上下文信息）、第8章（评分预测）
2. **B站王树森《推荐系统》公开课**：选看基础部分
   - 地址：https://www.bilibili.com/video/BV1oE411j7t8/
3. **阿里云天池推荐系统入门教程**：快速过一遍
   - 地址：https://tianchi.aliyun.com/course/336

补充参考资料

1. **GitHub开源推荐算法库Surprise**：快速过一遍，用Surprise实现协同过滤（选看）

- 地址：https://surprise.readthedocs.io/

必做实战任务

1. **推荐系统基础概念梳理**：
   - 写一篇笔记，梳理推荐系统的核心链路（召回→粗排→精排→重排）每个环节的作用
   - 写一篇笔记，梳理推荐系统的评价指标（离线+在线）
2. **传统推荐算法实现**：
   - 用MovieLens数据集实现**基于物品的协同过滤（Item-CF）**：
     - 计算物品之间的相似度（余弦相似度）
     - 给用户推荐Top-10物品
     - 用离线指标评估（准确率、召回率、RMSE）
   - 用MovieLens数据集实现**基于内容的推荐（CB）**：
     - 用电影的类型特征（One-Hot编码）计算物品相似度
     - 给用户推荐Top-10物品
     - 用离线指标评估
   - 用MovieLens数据集实现**基于逻辑回归的推荐（LR）**：
     - 用之前构建好的用户特征、电影特征、用户-电影交互特征
     - 把评分转化为二分类问题（评分≥3.5为正样本，否则为负样本，或者用点击/未点击作为标签）
     - 用scikit-learn的LR模型训练
     - 用离线指标评估（准确率、召回率、F1、AUC）
3. **算法对比**：对比Item-CF、CB、LR三个算法的离线指标，分析优缺点
4. **提交代码**：把传统推荐算法的实现代码提交到GitHub/Gitee仓库，写好[README.md](README.md)（说明算法原理、如何运行、评估结果）

通关标准

1. 能理解推荐系统的核心概念、全链路、评价指标
2. 能独立实现基于物品的协同过滤、基于内容的推荐、基于逻辑回归的推荐
3. 能用离线指标评估推荐算法
4. GitHub/Gitee仓库里有对应的算法实现代码

主线任务3：工业级深度学习推荐模型（2周）

学习重点

学习现在互联网公司主流的工业级深度学习推荐模型：

1. **深度学习基础（快速回顾，有基础可跳过）**：
    - 神经网络基础：感知机、激活函数（Sigmoid、ReLU、Tanh）、损失函数（交叉熵、MSE）、优化器（SGD、Adam）
    - PyTorch基础：张量、自动求导、模型定义、模型训练、模型保存与加载
2. **工业级深度学习推荐模型**：
    - **Wide&Deep**：Google提出，Wide部分处理记忆（Memorization），Deep部分处理泛化（Generalization）
    - **DeepFM**：Wide&Deep的改进，把Wide部分换成FM（因子分解机），自动学习特征交叉
    - **DIN（Deep Interest Network）**：阿里提出，用注意力机制（Attention）捕捉用户的动态兴趣
    - 这些模型的原理、实现、优缺点对比
3. **推荐模型训练流程**：
    - 数据准备：特征工程、训练集/验证集/测试集划分
    - 模型定义：用PyTorch定义模型
    - 模型训练：损失函数、优化器、训练循环、验证
    - 模型评估：离线指标（AUC、RMSE）
    - 模型保存：保存为PyTorch模型文件或TorchScript格式

学习方法

1. 先看《深度学习推荐系统》，理解工业级推荐模型的原理
2. 同步看B站的《深度学习推荐模型实战》视频，补充理解
3. 用PyTorch实现Wide&Deep、DeepFM模型，用MovieLens数据集训练和评估
4. 学习GitHub开源推荐算法库RecBole（选看，学有余力再看）

核心必学资料

1. **《深度学习推荐系统》**：王喆著，电子书/纸质书，工业界标杆书籍，必看
    - 重点看：第3章（深度学习在推荐系统中的应用）、第4章（Embedding与深度学习推荐系统）、第5章（深度学习推荐模型的进化）、第6章（Wide&Deep模型）、第7章（DeepFM模型）、第8章（DIN模型）
2. **PyTorch官方中文教程**：快速过一遍基础
    - 地址：https://pytorch.org/tutorials/beginner/basics/

3. **B站《深度学习推荐模型实战》**：选看Wide&Deep、DeepFM、DIN的实现部分
   - 地址：https://www.bilibili.com/video/BV1gK411G7zV/
4. **GitHub开源推荐算法库RecBole**：快速过一遍，了解工业级推荐算法库的实现（选看）
   - 地址：https://recbole.io/

补充参考资料

1. **Wide&Deep原论文**：https://arxiv.org/abs/1606.07792（选看，学有余力再看）
2. **DeepFM原论文**：https://arxiv.org/abs/1703.04247（选看）
3. **DIN原论文**：https://arxiv.org/abs/1706.06978（选看）

必做实战任务

1. **PyTorch基础练习**：
   - 用PyTorch定义一个简单的全连接神经网络，训练MNIST数据集（快速回顾，有基础可跳过）
   - 保存和加载PyTorch模型
2. **工业级推荐模型实现**：
   - 用PyTorch实现**Wide&Deep模型**：
     - 用之前构建好的MovieLens特征
     - Wide部分：用线性层处理稀疏特征（如用户ID、电影ID、类型）
     - Deep部分：用全连接层处理稠密特征（如用户历史点击次数、平均评分）
     - 拼接Wide和Deep的输出，用Sigmoid激活函数做二分类（CTR预测）
     - 训练模型，用AUC评估
   - 用PyTorch实现**DeepFM模型**：
     - 把Wide&Deep的Wide部分换成FM层
     - 其余部分和Wide&Deep类似
     - 训练模型，用AUC评估，和Wide&Deep对比
3. **模型保存**：把训练好的DeepFM模型保存为PyTorch模型文件（.pth）和TorchScript格式（.pt），供后续部署使用
4. **提交代码**：把深度学习推荐模型的实现代码提交到GitHub/Gitee仓库，写好[README.md](README.md)（说明模型原理、如何运行、评估结果）

通关标准

1. 能理解Wide&Deep、DeepFM、DIN的原理
2. 能独立用PyTorch实现Wide&Deep、DeepFM模型
3. 能训练和评估深度学习推荐模型

4. 能保存PyTorch模型为不同格式
5. GitHub/Gitee仓库里有对应的模型实现代码

主线任务4：向量数据库&大模型+推荐（1周）

学习重点

学习2026年推荐系统的核心方向：向量检索和大模型+推荐

1. **向量数据库Milvus**：
   - 向量检索基础：Embedding（向量）、相似度计算（余弦相似度、内积、L2距离）、近似最近邻搜索（ANN）
   - Milvus基础概念：Collection、Partition、Field、Vector、Index
   - Milvus安装与配置：本地安装Milvus（用Docker Compose）、Attu可视化工具
   - Milvus常用操作：创建Collection、插入向量、创建索引、搜索向量、删除Collection
   - Python/Go连接Milvus：pymilvus/milvus-sdk-go库
2. **大模型+推荐基础**：
   - 大模型在推荐系统中的应用场景：
     - 用户画像：用大模型理解用户的历史行为，生成更精准的用户画像
     - 内容理解：用大模型理解内容的文本、图片、视频，生成内容Embedding
     - 多模态推荐：结合文本、图片、视频的Embedding做推荐
     - 冷启动优化：用大模型生成新用户/新物品的Embedding
     - 个性化文案生成：用大模型生成推荐理由
   - LangChain基础：LangChain的核心概念、用LangChain连接大模型和向量数据库（基础）
   - 大模型Embedding：用OpenAI Embedding或开源Embedding模型（如BGE、M3E）生成文本Embedding

学习方法

1. 先看Milvus官方中文文档，学习向量检索和Milvus的使用
2. 本地安装Milvus，跟着文档做常用操作
3. 用Python/Go连接Milvus，实现向量检索
4. 学习LangChain基础，了解大模型+推荐的应用场景

核心必学资料

1. **Milvus官方中文文档**：全程跟着做

- 地址：https://milvus.io/docs/zh-CN
- 必看：
  - 快速开始：https://milvus.io/docs/zh-CN/install/standalone-docker.md
  - 用户指南：https://milvus.io/docs/zh-CN/manage_collections.md、https://milvus.io/docs/zh-CN/insert_data.md、https://milvus.io/docs/zh-CN/search.md
  - SDK参考：https://milvus.io/docs/zh-CN/install/sdk/pymilvus.md、https://milvus.io/docs/zh-CN/install/sdk/go.md

2. **LangChain官方中文文档**：快速过一遍基础
   - 地址：https://www.langchain.com.cn/docs
3. **OpenAI Embedding文档**：快速过一遍（选看，也可以用开源Embedding模型）
   - 地址：https://platform.openai.com/docs/guides/embeddings
4. **B站《大模型+推荐系统实战》**：选看基础部分

补充参考资料

1. **GitHub开源Embedding模型M3E**：中文开源Embedding模型，效果不错
   - 地址：https://github.com/wangyuxinwhy/uniem/tree/main/m3e
2. **Faiss官方文档**：Facebook开源的向量检索库，学有余力再看
   - 地址：https://faiss.ai/

必做实战任务

1. **Milvus基础练习**：
   - 本地安装Milvus和Attu（用Docker Compose）
   - 用Attu创建一个Collection（content），包含字段：content_id（INT64）、content_embedding（FLOAT_VECTOR，维度128）
   - 生成一些模拟的向量数据（用NumPy随机生成），插入Collection
   - 创建IVF_FLAT索引
   - 用一个模拟的查询向量，搜索Top-10相似的向量
2. **Milvus+MovieLens向量检索练习**：
   - 用之前训练好的DeepFM模型，提取电影的Embedding（或者用开源Embedding模型生成电影标题的Embedding）
   - 把电影ID和电影Embedding插入Milvus的Collection
   - 给一个用户喜欢的电影ID，从Milvus搜索Top-10相似的电影，作为推荐结果

- 用Python实现这个流程

3. **大模型+推荐基础练习**：
   - 用LangChain连接一个开源大模型（或OpenAI API）
   - 用大模型生成一段电影的描述，然后用Embedding模型生成描述的Embedding
   - 把Embedding插入Milvus，搜索相似的电影
4. **提交代码**：把Milvus向量检索的代码提交到GitHub/Gitee仓库，写好[README.md](README.md)

通关标准

1. 能理解向量检索的基础概念（Embedding、相似度计算、ANN）
2. 能熟练使用Milvus的常用操作（创建Collection、插入向量、创建索引、搜索向量）
3. 能用Python/Go连接Milvus，实现向量检索
4. 能理解大模型在推荐系统中的应用场景
5. GitHub/Gitee仓库里有对应的练习代码

主线任务5：模型部署&效果评估（1周）

学习重点

把训练好的推荐模型部署成可用的接口，学习推荐系统的效果评估：

1. **推荐模型部署**：
   - 用FastAPI封装PyTorch模型：把训练好的DeepFM模型封装成RESTful接口，接收用户ID和电影ID，返回预测的CTR或评分
   - 用TorchServe部署PyTorch模型：TorchServe是PyTorch官方的模型部署工具（选看）
   - 用Triton Inference Server部署模型：NVIDIA开源的推理部署工具，支持多框架（选看）
2. **推荐系统效果评估**：
   - 离线评估：用之前的指标（AUC、RMSE、准确率、召回率）
   - 在线A/B测试：
     - A/B测试的基础概念：实验组、对照组、流量分配、统计显著性
     - A/B测试的流程：确定实验目标、确定实验指标、设计实验、分配流量、上线实验、分析结果
     - A/B测试的指标：CTR、CVR、留存率、GMV、人均点击次数
   - 推荐系统的监控：实时监控推荐效果、模型性能、接口QPS和延迟

学习方法

1. 先把之前训练好的DeepFM模型用FastAPI封装成接口
2. 学习A/B测试的基础概念
3. 看一些推荐系统监控的资料

核心必学资料

1. **FastAPI官方文档**：回顾如何用FastAPI开发接口
2. **TorchServe官方文档**：快速过一遍（选看）
   - 地址：https://pytorch.org/serve/
3. **《A/B测试：实战指南》**：电子书/纸质书，选看基础部分
4. **阿里云A/B测试平台入门教程**：快速过一遍

补充参考资料

1. **Prometheus+Grafana官方文档**：监控工具，学有余力再看

必做实战任务

1. **推荐模型部署**：
   - 用FastAPI封装之前训练好的DeepFM模型：
     - 加载保存好的PyTorch模型
     - 创建一个接口：接收用户ID和电影ID（或一批电影ID）
     - 加载用户特征和电影特征
     - 用模型预测CTR
     - 返回预测结果（按CTR排序后的Top-10电影ID）
   - 启动FastAPI服务，用Swagger文档测试接口
   - 把这个服务打包成Docker镜像
2. **A/B测试基础练习**：
   - 写一篇笔记，设计一个推荐系统的A/B测试实验：
     - 实验目标：提升CTR
     - 实验指标：CTR、人均点击次数、留存率
     - 实验组：用DeepFM模型；对照组：用Item-CF算法
     - 流量分配：50%实验组，50%对照组
     - 实验时长：7天
     - 如何分析结果：统计显著性检验

3. **提交代码**：把模型部署的代码提交到GitHub/Gitee仓库，写好[README.md](README.md)（说明如何运行服务、如何测试接口）

通关标准

1. 能用FastAPI封装推荐模型，部署成RESTful接口
2. 能理解A/B测试的基础概念，能设计简单的A/B测试实验
3. 能理解推荐系统的监控指标
4. GitHub/Gitee仓库里有对应的部署代码

并行C++任务：C++模型推理加速+向量检索（2-3周）

学习重点

1. **PyTorch C++ API（LibTorch）**（1周）：
   - LibTorch基础：安装LibTorch、张量操作、自动求导（基础）
   - 加载PyTorch模型：加载TorchScript格式的模型（.pt文件）
   - 模型推理：用LibTorch做模型推理，输入数据，获取输出
2. **TensorRT C++ API**（1周）：
   - TensorRT基础概念：TensorRT的作用（模型量化、推理加速）、Engine、Context
   - 模型转换：把PyTorch模型转换为ONNX格式，再把ONNX模型转换为TensorRT Engine
   - TensorRT C++推理：加载TensorRT Engine，做推理
   - 性能对比：对比LibTorch和TensorRT的推理速度和精度
3. **Milvus C++ SDK**（0.5周）：
   - 用Milvus C++ SDK连接Milvus
   - 创建Collection、插入向量、搜索向量
4. **C++性能分析工具进阶**（0.5周）：
   - perf：更详细的性能分析（选看）
   - Valgrind：内存泄漏检测（选看）

学习方法

1. 先看LibTorch官方文档，学习用LibTorch加载和推理模型
2. 再看TensorRT官方文档，学习模型转换和推理加速
3. 用之前训练好的DeepFM模型做LibTorch和TensorRT的推理练习
4. 学习Milvus C++ SDK，实现向量检索

核心必学资料

1. **PyTorch C++ API官方文档**：全程跟着做
   - 地址：https://pytorch.org/tutorials/advanced/cpp_frontend.html
   - 必看：
     - Loading a TorchScript Model in C++：https://pytorch.org/tutorials/advanced/cpp_export.html
2. **TensorRT官方中文文档**：快速过一遍基础
   - 地址：https://docs.nvidia.com/deeplearning/tensorrt/index.html
   - 必看：
     - Quick Start Guide：https://docs.nvidia.com/deeplearning/tensorrt/quick-start-guide/index.html
     - Developer Guide：https://docs.nvidia.com/deeplearning/tensorrt/developer-guide/index.html
3. **Milvus C++ SDK文档**：全程跟着做
   - 地址：https://milvus.io/docs/zh-CN/install/sdk/cpp.md
4. **《Linux性能优化实战》（极客时间）**：选看「C++性能分析」模块（学有余力再看）

补充参考资料

1. **ONNX官方文档**：快速过一遍模型转换
   - 地址：https://onnx.ai/

必做实战任务

1. **LibTorch推理练习**：
   - 下载并安装LibTorch（CPU版本或GPU版本，根据你的电脑配置）
   - 把之前保存的DeepFM TorchScript模型（.pt文件）复制到C++项目目录
   - 用C++写一个LibTorch推理程序：
     - 加载TorchScript模型
     - 准备输入数据（用户特征、电影特征）
     - 用模型做推理
     - 输出预测结果
   - 编译并运行程序，测试推理结果是否和Python版本一致
2. **TensorRT推理练习**：
   - 把PyTorch DeepFM模型转换为ONNX格式（用torch.onnx.export）

- 用TensorRT的trtexec工具把ONNX模型转换为TensorRT Engine
- 用C++写一个TensorRT推理程序：
  - 加载TensorRT Engine
  - 准备输入数据
  - 做推理
  - 输出预测结果
- 对比LibTorch和TensorRT的推理速度（测1000次推理的平均时间）和精度

3. **Milvus C++ SDK练习**：
   - 用Milvus C++ SDK连接本地Milvus
   - 创建一个Collection，插入模拟的向量数据
   - 创建索引，搜索Top-10相似向量
4. **C++性能分析练习**：
   - 用gprof或perf分析上面的LibTorch推理程序的性能，找出瓶颈
5. **提交代码**：把C++模型推理和Milvus C++ SDK的练习代码提交到GitHub/Gitee仓库，写好[README.md](README.md)

通关标准

1. 能用LibTorch加载TorchScript模型并做推理
2. 能把PyTorch模型转换为ONNX格式，再转换为TensorRT Engine
3. 能用TensorRT C++ API做推理，对比LibTorch的性能
4. 能用Milvus C++ SDK连接Milvus并做向量检索
5. GitHub/Gitee仓库里有对应的练习代码

第四阶段：全链路项目整合&就业冲刺 + C++引擎整合（3-4周）

阶段目标

完成全链路推荐系统项目的整合与上线，准备面试，达到入门就业水平，同时把C++高性能组件整合进项目

主线任务1：全链路项目整合（2周）

项目名称

个性化内容/电影推荐系统（可根据你的兴趣选内容或电影）

项目架构（全链路）

1. **数据层**：
   - PostgreSQL：存储用户信息、内容信息、用户-内容交互信息
   - Redis：缓存用户画像、热门内容、限流
   - Kafka：采集用户行为日志（点击、浏览、收藏、评分）
   - Milvus：存储内容Embedding，做向量召回
   - Elasticsearch：存储内容信息，做关键词检索和过滤
2. **业务层**：
   - Python FastAPI：用户管理、内容管理、精排接口（用DeepFM模型）、业务逻辑
   - Go Gin：高并发召回接口（对接Milvus）、实时日志处理服务（Kafka消费者）、限流组件
3. **算法层**：
   - 特征工程：Python Pandas/NumPy，离线构建用户特征、内容特征
   - 召回：Milvus向量召回（Top-100）、Elasticsearch关键词召回（可选）
   - 精排：DeepFM模型（PyTorch），对召回结果排序（Top-10）
   - 重排：简单的规则重排（比如去重、调整热门内容的位置，可选）
4. **部署层**：
   - Docker：所有服务都打包成Docker镜像
   - Docker Compose：本地启动所有服务
   - 云服务器：项目部署到阿里云/腾讯云（可选，有条件的话）

项目功能

1. 用户端：
   - 用户注册、登录
   - 查看内容列表
   - 点击、浏览、收藏、评分内容
   - 查看个性化推荐内容（Top-10）
   - 搜索内容（关键词搜索）
2. 后端：
   - 实时采集用户行为日志到Kafka
   - 实时更新Redis里的用户画像
   - 召回接口：快速返回Top-100相似内容
   - 精排接口：用DeepFM模型排序，返回Top-10

- 精排接口：用DeepFM模型排序，返回Top-10
- 离线特征更新：每天定时更新用户特征、内容特征、内容Embedding
- 离线模型训练：每周定时重新训练DeepFM模型

项目实施步骤

1. **环境搭建**：
   - 用Docker Compose启动PostgreSQL、Redis、Kafka、Elasticsearch、Milvus、Attu
2. **数据层开发**：
   - 创建数据库表（用户表、内容表、交互表）
   - 导入模拟数据（或用MovieLens数据集）
   - 用Python做特征工程，构建用户特征、内容特征，保存到PostgreSQL
   - 用DeepFM模型提取内容Embedding，插入Milvus
   - 把内容信息插入Elasticsearch
3. **业务层开发**：
   - Python FastAPI：
     - 用户管理接口（注册、登录、信息查询）
     - 内容管理接口（内容列表、内容详情）
     - 精排接口：接收召回的Top-100内容ID，加载特征，用DeepFM模型预测CTR，排序后返回Top-10
     - 行为日志上报接口：接收用户行为，发送到Kafka
   - Go Gin：
     - 召回接口：接收用户ID，从Redis获取用户最近点击的内容ID，从Milvus搜索Top-100相似内容，返回
     - 实时日志处理服务：Kafka消费者，消费用户行为日志，更新Redis里的用户画像（最近点击的内容、点击次数）
     - 限流中间件：集成到召回接口，每个用户每分钟最多请求10次
4. **算法层整合**：
   - 把DeepFM模型部署成FastAPI接口，供业务层调用
   - 把召回接口和精排接口对接：用户请求推荐时，先调用Go的召回接口，再调用Python的精排接口，返回最终结果
5. **部署上线**：
   - 写Dockerfile，打包FastAPI服务、Go服务
   - 写docker-compose.yml，启动所有服务（PostgreSQL、Redis、Kafka、Elasticsearch、Milvus、FastAPI、Go）

- 本地测试所有功能
- 有条件的话，部署到云服务器，配置域名（可选）

6. **项目文档**：
   - 写[README.md](README.md)：说明项目架构、功能、技术栈、如何运行、接口文档
   - 写项目设计文档：架构图、数据库表结构、接口说明
   - 写项目总结：遇到的问题、解决方案、优化方向

核心必学资料

1. 之前所有阶段的资料，遇到问题随时回顾

必做实战任务

1. 完成上面的全链路项目整合
2. 本地测试所有功能，确保没有bug
3. 做压力测试：用ab或wrk压测召回接口和精排接口的QPS和延迟
4. 写好项目文档
5. 把项目代码提交到GitHub/Gitee仓库，设置为公开（方便面试时展示）

通关标准

1. 有完整的全链路推荐系统项目代码在GitHub/Gitee仓库
2. 项目能在本地用Docker Compose启动，所有功能正常
3. 有详细的项目文档（README.md、设计文档、总结）
4. 能清晰地讲解项目的架构、每个模块的作用、自己负责的部分

并行C++任务：C++高性能召回引擎整合（1-2周）

学习重点

把C++整合到全链路项目里，替换/优化核心组件：

1. **C++高性能向量召回引擎**：
   - 用C++重写之前的Go召回接口：
     - 用Milvus C++ SDK连接Milvus
     - 用Redis C++客户端（hiredis）连接Redis，获取用户最近点击的内容
     - 用C++写HTTP服务器（或用C++ Web框架，如Oat++、Crow），提供RESTful接口
     - 测试这个C++召回接口的QPS和延迟，和Go版本对比
2. **C++精排模型推理服务**（可选，学有余力再做）：

- 用TensorRT C++ API部署DeepFM模型
- 用C++写HTTP服务器，提供精排接口
- 对比Python版本的推理速度

3. **整合进全链路项目**：
   - 把C++召回引擎打包成Docker镜像
   - 修改docker-compose.yml，用C++召回引擎替换Go召回引擎
   - 测试全链路项目，确保功能正常
   - 做压力测试，对比C++版本和Go版本的性能

核心必学资料

1. 之前C++阶段的所有资料
2. **C++ Web框架Oat++官方文档**：选看（如果用C++写Web接口）
   - 地址：https://oatpp.io/

必做实战任务

1. 完成C++高性能向量召回引擎的开发
2. 把C++召回引擎整合进全链路项目
3. 做性能对比测试，记录QPS和延迟的提升
4. 更新项目文档，说明C++组件的作用和性能提升
5. 把C++组件的代码提交到GitHub/Gitee仓库

通关标准

1. C++召回引擎能正常工作，整合进全链路项目
2. 有性能对比测试结果，C++版本的QPS/延迟优于Go版本
3. 项目文档里有C++组件的说明

主线任务2：面试&就业准备（1-2周）

学习重点

1. **知识点梳理**：
   - 后端开发：Python/Go基础、FastAPI/Gin、PostgreSQL、Redis、Kafka、Elasticsearch、Docker
   - 推荐系统：基础概念、全链路、传统算法（Item-CF、CB、LR）、工业级深度学习模型（Wide&Deep、DeepFM、DIN）、向量检索（Milvus）、模型部署、A/B测试

- C++（可选，如果你想面C++优化岗）：现代C++、网络编程、LibTorch、TensorRT、性能优化

2. **简历优化**：
   - 简历结构：个人信息、教育背景、技能栈、项目经历、实习经历（如果有）、获奖经历（如果有）
   - 项目经历：用STAR法则（Situation、Task、Action、Result）描述，重点突出你做了什么、用了什么技术、取得了什么成果（比如QPS提升了多少、AUC提升了多少）
   - 技能栈：分熟练掌握、掌握、了解三个等级
3. **笔试刷题**：
   - 算法题：LeetCode简单+中等难度前50道（用Python或Go实现，面试常考）
   - SQL题：LeetCode SQL简单+中等难度前30道
   - 后端/推荐系统基础题：刷GitHub上的高频面试题
4. **面试准备**：
   - 自我介绍：准备1分钟、3分钟的自我介绍
   - 项目讲解：准备5-10分钟的项目讲解，能清晰地讲清楚项目架构、每个模块的作用、你负责的部分、遇到的问题和解决方案
   - 高频面试题：准备后端、推荐系统、C++的高频面试题的答案
   - 模拟面试：找朋友或用模拟面试平台练习

核心必学资料

1. **GitHub后端开发高频面试题**：
   - 地址：https://github.com/youngyangyang04/leetcode-master（算法题）
   - 地址：https://github.com/wolverinn/Waking-Up（后端基础题）
2. **GitHub推荐系统高频面试题**：
   - 地址：https://github.com/zhongqiangwu960812/AI-RecommenderSystem
3. **LeetCode**：
   - 地址：https://leetcode.cn/
4. **简历优化指南**：
   - 知乎、B站上的简历优化视频/文章

必做实战任务

1. **知识点梳理**：写笔记，梳理所有核心知识点
2. **简历优化**：写一份简历，重点突出全链路推荐系统项目
3. **笔试刷题**：

- 刷完LeetCode算法简单+中等前50道
- 刷完LeetCode SQL简单+中等前30道
- 刷完后端/推荐系统高频面试题

4. **面试准备**：
   - 准备自我介绍
   - 准备项目讲解
   - 准备高频面试题的答案
   - 模拟面试2-3次

通关标准

1. 有一份优化好的简历
2. 能清晰地梳理所有核心知识点
3. 能独立完成LeetCode算法和SQL的必刷题
4. 能流畅地做自我介绍和项目讲解
5. 能回答高频面试题

可选进阶：C++底层优化专项（学完主线后按需学习）

如果你的目标是**推荐系统底层引擎开发、模型推理加速专家**，可以继续学习以下内容：

1. **高性能并发编程**：
   - 《C++ Concurrency in Action》（第2版）：深入学习C++多线程编程
   - 无锁数据结构：学习无锁队列、无锁哈希表
   - 线程池：实现高性能线程池
2. **内存优化**：
   - 内存池：实现内存池，减少内存分配开销
   - 对象池：实现对象池，减少对象创建销毁开销
   - 缓存友好的代码设计：利用CPU缓存，提升性能
3. **推荐系统底层引擎**：
   - 学习Faiss向量检索库的C++源码：https://github.com/facebookresearch/faiss
   - 学习Milvus的C++底层源码：https://github.com/milvus-io/milvus
   - 用C++写一个简单的向量检索引擎（参考Faiss）
4. **实战任务**：

- 优化推荐系统的召回速度：用C++写一个更高效的向量检索引擎
- 优化推荐系统的精排速度：用TensorRT深度优化DeepFM模型
- 做性能对比测试，记录优化结果

学习注意事项

1. **每天坚持学习**：按每天2-3小时的节奏，不要中断，保持学习状态
2. **边学边练**：不要只看资料，一定要写代码练习，每学一个知识点就写对应的小代码
3. **多做项目**：项目是最好的学习方式，也是面试时最重要的展示，一定要完成全链路项目
4. **及时复习**：学完一个阶段后，及时复习之前的知识点，避免遗忘
5. **多交流**：加入技术社区（如GitHub、知乎、B站、技术群），和其他学习者交流，遇到问题及时提问
6. **保持耐心**：学习是一个循序渐进的过程，不要急于求成，遇到困难不要放弃

需要我把某个阶段的内容单独整理成一份**每日学习计划表**，或者给你某个项目的**代码框架**吗？

（注：文档部分内容可能由 AI 生成）